



L3B: Curves and Sound

CS1101S: Programming Methodology

Boyd Anderson

28th August, 2026

Version 1.0 — Built: 2026-08-28 01:56:47



Outline

Curves

Sound



Mastery Checks

- ▶ Oral tests



Mastery Checks

- ▶ Oral tests
- ▶ Conducted in 3-way meetings:
 - 2 students, 1 Avenger, mostly of the same Studio



Mastery Checks

- ▶ Oral tests
- ▶ Conducted in 3-way meetings:
 - 2 students, 1 Avenger, mostly of the same Studio
- ▶ Anytime during the semester



Mastery Checks

- ▶ Oral tests
- ▶ Conducted in 3-way meetings:
 - 2 students, 1 Avenger, mostly of the same Studio
- ▶ Anytime during the semester
- ▶ Topics for Mastery Check 1 (3% of total course grade)
 - Scope
 - Higher-order functions
 - Substitution model and iterative/recursive processes



Mastery Checks

- ▶ Oral tests
- ▶ Conducted in 3-way meetings:
 - 2 students, 1 Avenger, mostly of the same Studio
- ▶ Anytime during the semester
- ▶ Topics for Mastery Check 1 (3% of total course grade)
 - Scope
 - Higher-order functions
 - Substitution model and iterative/recursive processes
- ▶ Pass/fail for each topic
 - Repetition allowed per topic



Mastery Checks: The Process

- ▶ Form pairs. Your Avenger will help you find a partner from a different Studio if necessary



Mastery Checks: The Process

- ▶ Form pairs. Your Avenger will help you find a partner from a different Studio if necessary
- ▶ Make appointment with Avenger



Mastery Checks: The Process

- ▶ Form pairs. Your Avenger will help you find a partner from a different Studio if necessary
- ▶ Make appointment with Avenger
- ▶ For each topic, you have 10 minutes to convince the Avenger that both of you have mastered the topic



Mastery Checks: The Process

- ▶ Form pairs. Your Avenger will help you find a partner from a different Studio if necessary
- ▶ Make appointment with Avenger
- ▶ For each topic, you have 10 minutes to convince the Avenger that both of you have mastered the topic
- ▶ Use proper English and terminology; illustrate your argument with examples



Mastery Checks: The Process

- ▶ Form pairs. Your Avenger will help you find a partner from a different Studio if necessary
- ▶ Make appointment with Avenger
- ▶ For each topic, you have 10 minutes to convince the Avenger that both of you have mastered the topic
- ▶ Use proper English and terminology; illustrate your argument with examples
- ▶ Be ready for using examples given by the Avenger



Mastery Checks: The Process

- ▶ Form pairs. Your Avenger will help you find a partner from a different Studio if necessary
- ▶ Make appointment with Avenger
- ▶ For each topic, you have 10 minutes to convince the Avenger that both of you have mastered the topic
- ▶ Use proper English and terminology; illustrate your argument with examples
- ▶ Be ready for using examples given by the Avenger
- ▶ Be ready to answer questions on the topic posed by Avenger



Mastery Checks: The Process

- ▶ Form pairs. Your Avenger will help you find a partner from a different Studio if necessary
- ▶ Make appointment with Avenger
- ▶ For each topic, you have 10 minutes to convince the Avenger that both of you have mastered the topic
- ▶ Use proper English and terminology; illustrate your argument with examples
- ▶ Be ready for using examples given by the Avenger
- ▶ Be ready to answer questions on the topic posed by Avenger
- ▶ Avenger will pass you on a topic only if s/he is convinced you've mastered it



Mastery Checks: The Process

- ▶ Form pairs. Your Avenger will help you find a partner from a different Studio if necessary
- ▶ Make appointment with Avenger
- ▶ For each topic, you have 10 minutes to convince the Avenger that both of you have mastered the topic
- ▶ Use proper English and terminology; illustrate your argument with examples
- ▶ Be ready for using examples given by the Avenger
- ▶ Be ready to answer questions on the topic posed by Avenger
- ▶ Avenger will pass you on a topic only if s/he is convinced you've mastered it
- ▶ The pair can have repeat attempt(s) with the Avenger on unsuccessful topic(s)



Mastery Checks: The Process

- ▶ Form pairs. Your Avenger will help you find a partner from a different Studio if necessary
- ▶ Make appointment with Avenger
- ▶ For each topic, you have 10 minutes to convince the Avenger that both of you have mastered the topic
- ▶ Use proper English and terminology; illustrate your argument with examples
- ▶ Be ready for using examples given by the Avenger
- ▶ Be ready to answer questions on the topic posed by Avenger
- ▶ Avenger will pass you on a topic only if s/he is convinced you've mastered it
- ▶ The pair can have repeat attempt(s) with the Avenger on unsuccessful topic(s)
- ▶ Other staff may check as well for repeated attempts



Mastery Checks: The Process

- ▶ Form pairs. Your Avenger will help you find a partner from a different Studio if necessary
- ▶ Make appointment with Avenger
- ▶ For each topic, you have 10 minutes to convince the Avenger that both of you have mastered the topic
- ▶ Use proper English and terminology; illustrate your argument with examples
- ▶ Be ready for using examples given by the Avenger
- ▶ Be ready to answer questions on the topic posed by Avenger
- ▶ Avenger will pass you on a topic only if s/he is convinced you've mastered it
- ▶ The pair can have repeat attempt(s) with the Avenger on unsuccessful topic(s)
- ▶ Other staff may check as well for repeated attempts

Try to finish MC1 before Recess Week (great prep for Midterm!)



Logical Operators

- ▶ `not p`: true when `p` is false



Logical Operators

- ▶ `not p`: true when `p` is false
- ▶ `p and q`: true when both `p` and `q` are true



Logical Operators

- ▶ `not p`: true when `p` is false
- ▶ `p and q`: true when both `p` and `q` are true
- ▶ `p or q`: true when at least one of `p`, `q` is true

Logical Operators

- ▶ `not p`: true when `p` is false
- ▶ `p and q`: true when both `p` and `q` are true
- ▶ `p or q`: true when at least one of `p`, `q` is true

`and/or` are syntactic forms, not operators

Unlike a real binary operator, the right-hand expression is not always evaluated ([SICPy 1.1.6](#)):

`p and q` $\equiv$ `q` if `p` else `False`

`p or q` $\equiv$ `True` if `p` else `q`



Logical Operators: Laziness

```
def boom():  
    return 1 / 0  # would crash if ever evaluated  
  
print(False and boom())  
print(True or boom())
```





Logical Operators: Laziness

```
def boom():  
    return 1 / 0  # would crash if ever evaluated  
  
print(False and boom())  
print(True or boom())
```



Neither call to `boom()` ever executes — `and/or` stop as soon as the result is decided.



Outline

Curves

Specifications of Curves in Python

Sound



Path, Missions & Quests on Curves

- ▶ Path: “Curves”



Path, Missions & Quests on Curves

- ▶ Path: “Curves”
- ▶ Mission: “Curve Introduction”



Path, Missions & Quests on Curves

- ▶ Path: “Curves”
- ▶ Mission: “Curve Introduction”
- ▶ Mission: “Curve Manipulation”



Path, Missions & Quests on Curves

- ▶ Path: “Curves”
- ▶ Mission: “Curve Introduction”
- ▶ Mission: “Curve Manipulation”
- ▶ Mission: “Beyond the First Dimension”



Path, Missions & Quests on Curves

- ▶ Path: “Curves”
- ▶ Mission: “Curve Introduction”
- ▶ Mission: “Curve Manipulation”
- ▶ Mission: “Beyond the First Dimension”
- ▶ Quest: “Cardioid Arrest”



Path, Missions & Quests on Curves

- ▶ Path: “Curves”
- ▶ Mission: “Curve Introduction”
- ▶ Mission: “Curve Manipulation”
- ▶ Mission: “Beyond the First Dimension”
- ▶ Quest: “Cardioid Arrest”
- ▶ Quest: “Curvaceous Wizardry”



Path, Missions & Quests on Curves

- ▶ Path: “Curves”
- ▶ Mission: “Curve Introduction”
- ▶ Mission: “Curve Manipulation”
- ▶ Mission: “Beyond the First Dimension”
- ▶ Quest: “Cardioid Arrest”
- ▶ Quest: “Curvaceous Wizardry”
- ▶ Contest: “The Choreographer”



Representation of Curves — Explicit Form

- ▶ Curves in 2D



Representation of Curves — Explicit Form

- ▶ Curves in 2D

The value of the dependent variable is given in terms of the independent variable $y = f(x)$



Representation of Curves — Explicit Form

- ▶ Curves in 2D

The value of the dependent variable is given in terms of the independent variable $y = f(x)$

Example: $y = mx + h$ (straight line)



Representation of Curves — Explicit Form

- ▶ Curves in 2D
The value of the dependent variable is given in terms of the independent variable $y = f(x)$
Example: $y = mx + h$ (straight line)
- ▶ Some curves cannot be expressed in explicit form



Representation of Curves — Explicit Form

- ▶ Curves in 2D
The value of the dependent variable is given in terms of the independent variable $y = f(x)$
Example: $y = mx + h$ (straight line)
- ▶ Some curves cannot be expressed in explicit form
Examples: vertical straight line, circle



Representation of Curves — Explicit Form

- ▶ Curves in 2D
The value of the dependent variable is given in terms of the independent variable $y = f(x)$
Example: $y = mx + h$ (straight line)
- ▶ Some curves cannot be expressed in explicit form
Examples: vertical straight line, circle
- ▶ Curves in 3D



Representation of Curves — Explicit Form

- ▶ Curves in 2D
The value of the dependent variable is given in terms of the independent variable $y = f(x)$
Example: $y = mx + h$ (straight line)
- ▶ Some curves cannot be expressed in explicit form
Examples: vertical straight line, circle
- ▶ Curves in 3D
Requires two equations

Representation of Curves — Explicit Form

- ▶ Curves in 2D
The value of the dependent variable is given in terms of the independent variable $y = f(x)$
Example: $y = mx + h$ (straight line)
- ▶ Some curves cannot be expressed in explicit form
Examples: vertical straight line, circle
- ▶ Curves in 3D
Requires two equations
 - $y = f(x)$



Representation of Curves — Explicit Form

- ▶ Curves in 2D
The value of the dependent variable is given in terms of the independent variable $y = f(x)$
Example: $y = mx + h$ (straight line)
- ▶ Some curves cannot be expressed in explicit form
Examples: vertical straight line, circle
- ▶ Curves in 3D
Requires two equations
 - $y = f(x)$
 - $z = g(x)$



Representation of Curves — Implicit Form

- ▶ Curves in 2D



Representation of Curves — Implicit Form

- ▶ Curves in 2D
 $f(x, y) = 0$

Representation of Curves — Implicit Form

► Curves in 2D

$$f(x, y) = 0$$

Examples:

- $ax + by + c = 0$ (straight line)
- $x^2 + y^2 - r^2 = 0$ (circle)

Representation of Curves — Implicit Form

► Curves in 2D

$$f(x, y) = 0$$

Examples:

- $ax + by + c = 0$ (straight line)
- $x^2 + y^2 - r^2 = 0$ (circle)

► Curves in 3D

Representation of Curves — Implicit Form

► Curves in 2D

$$f(x, y) = 0$$

Examples:

- $ax + by + c = 0$ (straight line)
- $x^2 + y^2 - r^2 = 0$ (circle)

► Curves in 3D

Can be represented as the intersection of two surfaces

Representation of Curves — Implicit Form

► Curves in 2D

$$f(x, y) = 0$$

Examples:

- $ax + by + c = 0$ (straight line)
- $x^2 + y^2 - r^2 = 0$ (circle)

► Curves in 3D

Can be represented as the intersection of two surfaces

- $f(x, y, z) = 0$
- $g(x, y, z) = 0$

Representation of Curves — Implicit Form

► Curves in 2D

$$f(x, y) = 0$$

Examples:

- $ax + by + c = 0$ (straight line)
- $x^2 + y^2 - r^2 = 0$ (circle)

► Curves in 3D

Can be represented as the intersection of two surfaces

- $f(x, y, z) = 0$
- $g(x, y, z) = 0$

► A drawback: Difficult to obtain points on the curves

Representation of Curves — Implicit Form

► Curves in 2D

$$f(x, y) = 0$$

Examples:

- $ax + by + c = 0$ (straight line)
- $x^2 + y^2 - r^2 = 0$ (circle)

► Curves in 3D

Can be represented as the intersection of two surfaces

- $f(x, y, z) = 0$
- $g(x, y, z) = 0$

- A drawback: Difficult to obtain points on the curves
Because the equations are just membership tests



Representation of Curves — Parametric Form

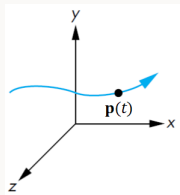
- ▶ Curves in 2D and 3D

Representation of Curves — Parametric Form

- Curves in 2D and 3D

Each spatial variable for points on the curve is expressed in terms of an independent variable t , the parameter

$$\mathbf{p}(t) = \begin{bmatrix} x(t) \\ y(t) \end{bmatrix} \quad \mathbf{p}(t) = \begin{bmatrix} x(t) \\ y(t) \\ z(t) \end{bmatrix}$$



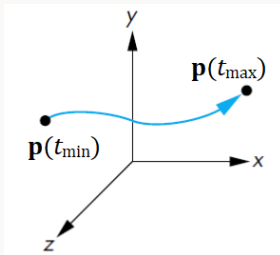


Representation of Curves — Parametric Form (cont'd)

- ▶ A curve segment is defined for $t_{\min} \leq t \leq t_{\max}$

Representation of Curves — Parametric Form (cont'd)

- ▶ A curve segment is defined for $t_{\min} \leq t \leq t_{\max}$
Often, $0 \leq t \leq 1$



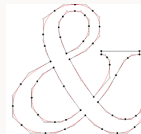


Applications of Curves

- ▶ Drawing smooth, vector-based curves at any resolution

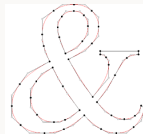
Applications of Curves

- ▶ Drawing smooth, vector-based curves at any resolution
- ▶ Font design and rendering



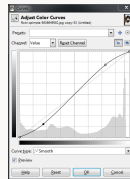
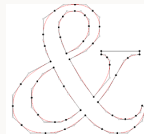
Applications of Curves

- ▶ Drawing smooth, vector-based curves at any resolution
- ▶ Font design and rendering
- ▶ Smooth animation paths (objects, lights, cameras)



Applications of Curves

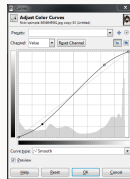
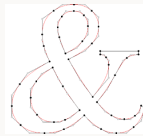
- ▶ Drawing smooth, vector-based curves at any resolution
- ▶ Font design and rendering
- ▶ Smooth animation paths (objects, lights, cameras)
- ▶ Designing smooth functions
 - E.g. for image color & tone adjustment





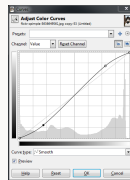
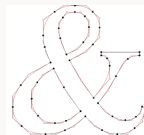
Applications of Curves

- ▶ Drawing smooth, vector-based curves at any resolution
- ▶ Font design and rendering
- ▶ Smooth animation paths (objects, lights, cameras)
- ▶ Designing smooth functions
 - E.g. for image color & tone adjustment
- ▶ 3D model design and rendering
 - Engineering and industrial design, movies, and games



Applications of Curves

- ▶ Drawing smooth, vector-based curves at any resolution
- ▶ Font design and rendering
- ▶ Smooth animation paths (objects, lights, cameras)
- ▶ Designing smooth functions
 - E.g. for image color & tone adjustment
- ▶ 3D model design and rendering
 - Engineering and industrial design, movies, and games
- ▶ Data fitting





Curves in Python

- ▶ Supported by the curve module
<https://source-academy.github.io/modules/documentation/modules/curve.html>



Curves in Python

- ▶ Supported by the curve module
<https://source-academy.github.io/modules/documentation/modules/curve.html>
- ▶ Uses parametric representation



Curves in Python

- ▶ Supported by the curve module
<https://source-academy.github.io/modules/documentation/modules/curve.html>
- ▶ Uses parametric representation
- ▶ Parameter t is within the unit interval $[0, 1]$
 - If C is a Curve, its
 - starting point is $C(0)$
 - ending point is $C(1)$



Specifications of Curves in Python

Curve

$\text{Curve} := \text{Number} \rightarrow \text{Point}$

A **Curve** is a **function** that takes a **Number** as argument and returns a **Point**. The **Number** argument must be within the interval $[0, 1]$.

Point

$\text{make_point} : (\text{Number}, \text{Number}) \rightarrow \text{Point}$

$\text{x_of}, \text{y_of} : \text{Point} \rightarrow \text{Number}$

$\text{x_of}(\text{make_point}(n, m)) = n$

$\text{y_of}(\text{make_point}(n, m)) = m$



Defining Curves

► Example:

```
def diagonal_line(t):  
    return make_point(t, t)  
  
draw_connected(10)(diagonal_line)
```





What does `draw_connected` do?

► Example:

```
def print_point(p):  
    print("point:", x_of(p), y_of(p))  
  
def diagonal_line(t):  
    p = make_point(t, t)  
    print_point(p)  
    return p  
  
draw_connected(10)(diagonal_line)
```




Let's make a general line!

► First Try

```
def line_with(m, h, t):  
    return make_point(t, m * t + h)  
  
draw_connected(10)(line_with)
```





Attempt 2:

► Second Try

```
def line_with(m, h):  
    return lambda t: make_point(t, m * t + h)  
  
draw_connected(10)(line_with(2, 0))
```





Off the map!

► Second Try

```
def line_with(m, h):  
    return lambda t: make_point(t, m * t + h)  
  
draw_connected(10)(line_with(2, 3))
```





λ

A more complex example:

► Example:

A more complex example:

► Example:

```
def unit_circle(t):  
    return make_point(math.cos(2 * math.pi * t),  
                      math.sin(2 * math.pi * t))
```

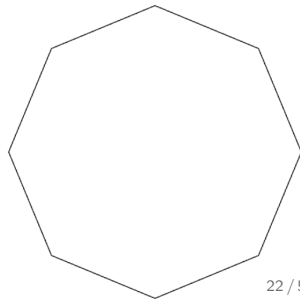
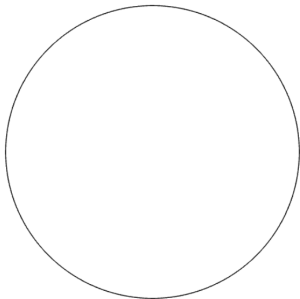




Drawing Curves

► Examples:

```
draw_connected(200)(unit_circle)  
draw_connected_full_view_proportional(200)(unit_circle)  
draw_connected_full_view_proportional(8)(unit_circle)
```





Example: Spiral Curves

- ▶ Wanted: Write a function `spiral_one` that represents a spiral curve
 - Uses `unit_circle`
 - One revolution only



Example: Spiral Curves

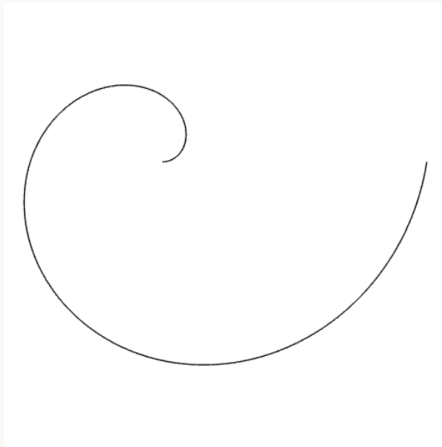
- ▶ Wanted: Write a function `spiral_one` that represents a spiral curve
 - Uses `unit_circle`
 - One revolution only
- ▶ Solution:

Example: Spiral Curves

- ▶ Wanted: Write a function `spiral_one` that represents a spiral curve
 - Uses `unit_circle`
 - One revolution only
- ▶ Solution:

```
def spiral_one(t):  
    p = unit_circle(t)  
    return make_point(t * x_of(p), t * y_of(p))  
  
draw_connected_full_view_proportional(200)(spiral_one)
```

Example: Spiral Curves (Result)





Example: Spiral Curves

- ▶ Wanted: Write a function `spiral` that returns a spiral curve
 - Uses `unit_circle`
 - Number of revolutions is a parameter



Example: Spiral Curves

- ▶ Wanted: Write a function `spiral` that returns a spiral curve
 - Uses `unit_circle`
 - Number of revolutions is a parameter
- ▶ Attempt #1:

Example: Spiral Curves

- ▶ Wanted: Write a function `spiral` that returns a spiral curve
 - Uses `unit_circle`
 - Number of revolutions is a parameter
- ▶ Attempt #1:

```
def spiral(rev, t):  
    p = unit_circle((t * rev) % 1)  
    return make_point(t * x_of(p), t * y_of(p))  
  
draw_connected_full_view_proportional(200)(spiral)
```



Example: Spiral Curves

- ▶ Wanted: Write a function `spiral` that returns a spiral curve
 - Uses `unit_circle`
 - Number of revolutions is a parameter



Example: Spiral Curves

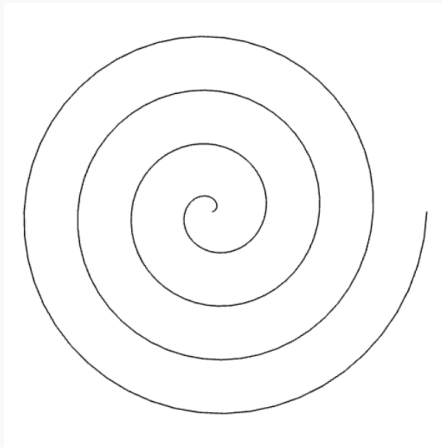
- ▶ Wanted: Write a function `spiral` that returns a spiral curve
 - Uses `unit_circle`
 - Number of revolutions is a parameter
- ▶ Attempt #2:

Example: Spiral Curves

- ▶ Wanted: Write a function spiral that returns a spiral curve
 - Uses unit_circle
 - Number of revolutions is a parameter
- ▶ Attempt #2:

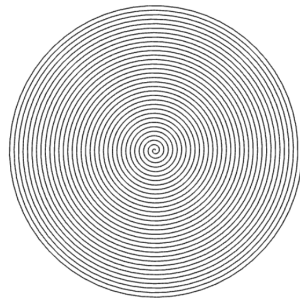
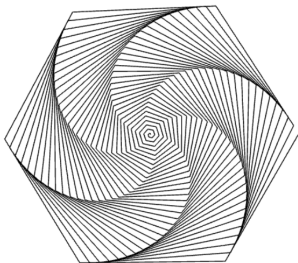
```
def spiral(rev):  
    def spiral_helper(t):  
        p = unit_circle((t * rev) % 1)  
        return make_point(t * x_of(p), t * y_of(p))  
    return spiral_helper  
  
draw_connected_full_view_proportional(200)(spiral(4))
```


Example: Spiral Curves (Result)



Example: Spiral Curves

```
draw_connected_full_view_proportional(200)(spiral(33))  
draw_connected_full_view_proportional(2000)(spiral(33))
```



Transformations on Curves

- ▶ Rotate `unit_line` by $\pi/6$, then translate it upwards by 0.25.

Transformations on Curves

- ▶ Rotate `unit_line` by $\pi/6$, then translate it upwards by 0.25.

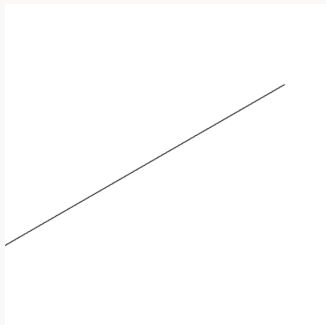
```
rot_line = rotate_around_origin(math_pi / 6)(unit_line)
shifted_rot_line = translate(0, 0.25, 0)(rot_line)
draw_connected(200)(shifted_rot_line)
```



Transformations on Curves

- Rotate `unit_line` by $\pi/6$, then translate it upwards by 0.25.

```
rot_line = rotate_around_origin(math_pi / 6)(unit_line)
shifted_rot_line = translate(0, 0.25, 0)(rot_line)
draw_connected(200)(shifted_rot_line)
```





Transformations on Curves

- ▶ Alternatively, compose the transformations before applying them to the curve.



Transformations on Curves

- ▶ Alternatively, compose the transformations before applying them to the curve.

```
def compose(f, g):  
    return lambda x: f(g(x))
```



Transformations on Curves

- Alternatively, compose the transformations before applying them to the curve.

```
def compose(f, g):  
    return lambda x: f(g(x))
```

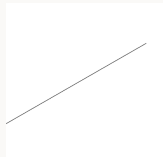
```
shift_rot = compose(translate(0, 0.25, 0),  
                    rotate_around_origin(math.pi / 6))  
shifted_rot_line = shift_rot(unit_line)  
draw_connected(200)(shifted_rot_line)
```


Transformations on Curves

- ▶ Alternatively, compose the transformations before applying them to the curve.

```
def compose(f, g):  
    return lambda x: f(g(x))
```

```
shift_rot = compose(translate(0, 0.25, 0),  
                    rotate_around_origin(math.pi / 6))  
shifted_rot_line = shift_rot(unit_line)  
draw_connected(200)(shifted_rot_line)
```





Connecting Curves

► Example:

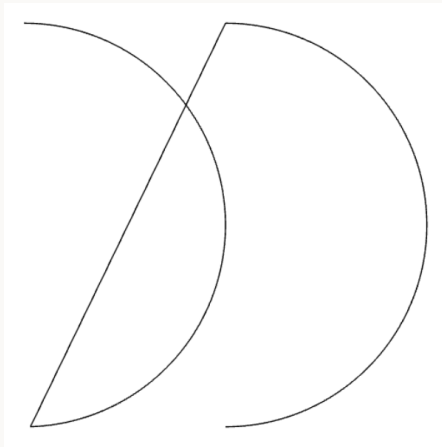


Connecting Curves

► Example:

```
def connect_rigidly(curve1, curve2):  
    return lambda t: (  
        curve1(2 * t)  
        if t < 1 / 2  
        else curve2(2 * t - 1)  
    )  
  
result_curve = connect_rigidly(arc, translate(1, 0,  
0)(arc))
```

Connecting Curves (Result)





λ

Colored Curves

► Example:



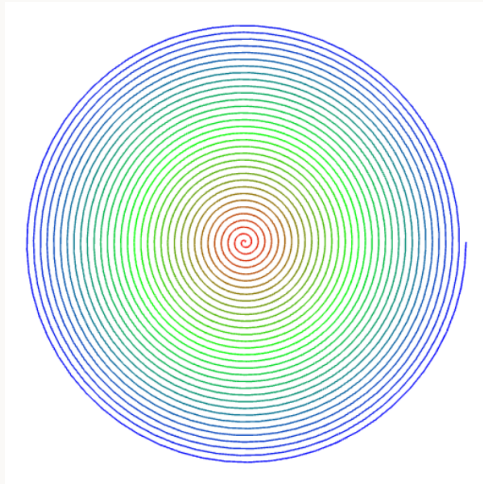
Colored Curves

► Example:

```
def colorful_spiral(rev):  
    def spiral_helper(t):  
        p = unit_circle((t * rev) % 1)  
        R = max(0, 1 - 2 * t) * 255  
        G = (1 - abs(1 - 2 * t)) * 255  
        B = max(0, 2 * t - 1) * 255  
        return make_color_point(t * x_of(p),  
                                t * y_of(p),  
                                R, G, B)  
    return spiral_helper
```



Colored Curves (Result)





3D Curves

► Example:



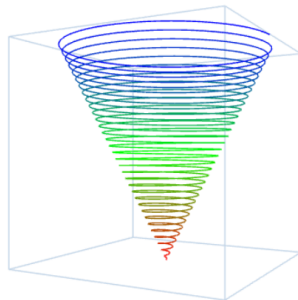
3D Curves

► Example:

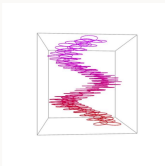
```
def colorful_3D_spiral(rev):  
    def spiral_helper(t):  
        p = unit_circle((t * rev) % 1)  
        R = max(0, 1 - 2 * t) * 255  
        G = (1 - abs(1 - 2 * t)) * 255  
        B = max(0, 2 * t - 1) * 255  
        return make_3D_color_point(  
            t * x_of(p), t * y_of(p), 2 * t, R, G, B)  
    return spiral_helper
```

λ

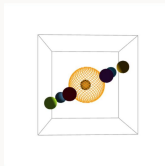
3D Curves



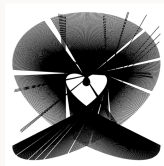
Curves from Past Contests



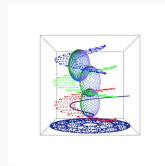
[Link to code](#)



[Link to code](#)



[Link to code](#)



[Link to code](#)



Outline

Curves

Sound

- What is Sound?

- Digital Sound

- Specifications of Sound in Python



Missions & Quests on Sound

- ▶ Mission: “Premorseal Communications”



Missions & Quests on Sound

- ▶ Mission: “Premorseal Communications”
- ▶ Mission: “POTS and Pans”



Missions & Quests on Sound

- ▶ Mission: “Premorseal Communications”
- ▶ Mission: “POTS and Pans”
- ▶ Mission: “Musical Diversions”



Missions & Quests on Sound

- ▶ Mission: “Premorseal Communications”
- ▶ Mission: “POTS and Pans”
- ▶ Mission: “Musical Diversions”
- ▶ Quest: “Echoes of the Past”



Missions & Quests on Sound

- ▶ Mission: “Premorseal Communications”
- ▶ Mission: “POTS and Pans”
- ▶ Mission: “Musical Diversions”
- ▶ Quest: “Echoes of the Past”
- ▶ Quest: “The Magical Tone Matrix”



Missions & Quests on Sound

- ▶ Mission: “Premorseal Communications”
- ▶ Mission: “POTS and Pans”
- ▶ Mission: “Musical Diversions”
- ▶ Quest: “Echoes of the Past”
- ▶ Quest: “The Magical Tone Matrix”
- ▶ Contest: “Game of Tones”



Curves

Sound

What is Sound?

Digital Sound

Specifications of Sound in Python



Curves

Sound

What is Sound?

Digital Sound

Specifications of Sound in Python



Specifications of Sound in Python

Wave

$\text{Wave} := \text{Number} \rightarrow \text{Number}$

A Wave is a **function** that takes a **Number** (time, in seconds) as argument and returns a **Number** (amplitude, between -1 and 1).

Sound

$\text{make_sound} : (\text{Wave}, \text{Number}) \rightarrow \text{Sound}$



A Simple Example

- ▶ Simple example:



A Simple Example

► Simple example:

```
pitch_A_wave = lambda t: math.sin(2 * math.pi * 440 * t)
pitch_A = make_sound(pitch_A_wave, 1.5)
play(pitch_A)
```

How does make_sound work?

► Simple example:

```
def pitch_A_wave(t):  
    print("current t:", t)  
    return math.sin(2 * math.pi * 440 * t)  
  
pitch_A = make_sound(pitch_A_wave, 0.1)  
play(pitch_A)
```




C Major Chord

- ▶ Second example:

C Major Chord

► Second example:

```
C_maj_chord_wave = lambda t: (  
    0.33 * math.sin(2 * math.pi * 261.63 * t) +  
    0.33 * math.sin(2 * math.pi * 329.63 * t) +  
    0.33 * math.sin(2 * math.pi * 392.00 * t)  
)  
C_maj_chord = make_sound(C_maj_chord_wave, 1.5)  
play(C_maj_chord)
```



λ

Do, a deer, a female deer

- ▶ Third example:



Do, a deer, a female deer

► Third example:

```
doremi_wave = lambda t: (  
    math.sin(2 * math.pi * 261.63 * t)  
    if t < 0.5  
    else math.sin(2 * math.pi * 293.66 * t)  
    if t < 1.0  
    else math.sin(2 * math.pi * 329.63 * t)  
)  
doremi = make_sound(doremi_wave, 1.5)  
play(doremi)
```



Sound in Python

- ▶ Supported by the sound module
<https://source-academy.github.io/modules/documentation/modules/sound.html>

Uses functional sound!



Sample Songs

Mission Impossible

```
play_in_tab(mission)
```



Minecraft Theme

```
play_in_tab(minecraft_theme)
```



λ

Testing... Testing... 1, 2, 3...



Testing... Testing... 1, 2, 3...

Time for a Sound Check(-ln)!



Testing... Testing... 1, 2, 3...

Time for a Sound Check(-In)!

Look for Check-In I3B in the Source Academy!



Summary

- ▶ We can create curves by defining a function that maps from a parameter to a Point.



Summary

- ▶ We can create curves by defining a function that maps from a parameter to a Point.
- ▶ The curve is drawn by sampling our function and generating points and interpolating between them.



Summary

- ▶ We can create curves by defining a function that maps from a parameter to a Point.
- ▶ The curve is drawn by sampling our function and generating points and interpolating between them.
- ▶ We can also create sounds by defining a function that describes the sound over time.



Summary

- ▶ We can create curves by defining a function that maps from a parameter to a Point.
- ▶ The curve is drawn by sampling our function and generating points and interpolating between them.
- ▶ We can also create sounds by defining a function that describes the sound over time.
- ▶ The audio is synthesized by sampling our function and generating points and then interpolating between them.



Summary

- ▶ We can create curves by defining a function that maps from a parameter to a Point.
- ▶ The curve is drawn by sampling our function and generating points and interpolating between them.
- ▶ We can also create sounds by defining a function that describes the sound over time.
- ▶ The audio is synthesized by sampling our function and generating points and then interpolating between them.
- ▶ We saw the sound and curve modules in the Source Academy!



Summary

- ▶ We can create curves by defining a function that maps from a parameter to a Point.
- ▶ The curve is drawn by sampling our function and generating points and interpolating between them.
- ▶ We can also create sounds by defining a function that describes the sound over time.
- ▶ The audio is synthesized by sampling our function and generating points and then interpolating between them.
- ▶ We saw the sound and curve modules in the Source Academy!
- ▶ There are contests coming up that use these!



Summary

- ▶ We can create curves by defining a function that maps from a parameter to a Point.
- ▶ The curve is drawn by sampling our function and generating points and interpolating between them.
- ▶ We can also create sounds by defining a function that describes the sound over time.
- ▶ The audio is synthesized by sampling our function and generating points and then interpolating between them.
- ▶ We saw the sound and curve modules in the Source Academy!
- ▶ There are contests coming up that use these!

Unit 1 is officially over woohooo! (except for S4 and RA1)